

Assignment 1: MCP Fundamentals

Theory Questions

Question 1

Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.

Answer:

In MCP, tools, resources, and prompts are three different primitives used to connect an AI application with an MCP server.

A tool is a function that the model calls to perform an action. It can take input and give the output.

For example, in the MCP hands-on we created two tools one to add the learnings to GitHub and search for the learnings using the keywords from the learnings.

A resource provides the data or info to the AI. It is mainly used to give context.

For example, in the MCP hands-on, the resource is the place where we are storing all the saved learnings as JSON format.

A prompt is reusable instruction template, that can be used by user to guide the model for a task.

For example, we have made a `summarize_week()`, which is a reusable instruction that asks the LLM to summarize the learnings by week.

Question 2

A local MCP server usually talks to Claude Desktop over stdio instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?

Answer:

First of all stdio means standard input and standard output.

Instead of communicating through a network HTTP, the MCP client starts a MCP server as a process and communicates with it through input and output.

As in the example:

Claude Desktop → stdin → MCP Server → stdout → Claude Desktop

So this makes sense for a local MCP because both client and server are running in a same computer. There is no need to expose a network API.

It is easy to setup.

FastMCP Documentation Questions

Question 1

FastMCP supports three transports: stdio, http, and sse. Read the deployment docs and explain when you'd pick stdio vs http, and what actually changes in your `mcp.run()` call to switch between them.

Answer:

Fast mcp is a python framework used to build mcp servers easily.

In it stdio is used when the server runs locally and the clients starts the server's process and communication through standard i/p and o/p.

http is useful when the server needs to run as a web server and to be accessed over network. It is suitable where clients connect through a http endpoint.

And main change is the transport argument in `mcp.run()`.

```
mcp.run(transport='stdio ')
```

```
mcp.run(transport='streamable-http ')
```

Question 2

FastMCP automatically turns some tool return values into “structured output” and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.

Answer:

So basically when a fastmcp tool return s a plain string, the result is treated as text output. The client will receives it human readable format.

For example:

```
return "rohit sharma is a goated batsman"
```

It will only return the above line.

When a tool will return a dictionary, fastmcp will provide the result in a structured way. The client can access individual fields instead of interpret everything.

For example:

```
return {
    "name": "rohit sharma",
    "team": "India",
    "no": "45"
}
```

It will return a structured format output.

So, the structured output matters when the clients needs to process the returned data. Or else in a plain text they need to interpret first.